Gemini

Danyang Zhuo
Duke University

Some materials from Zhuang's Gemini Slides

Large models

Characteristics

- Recent large language models (LLMs)

Model	Parameters	Accelerators	Training time	Developer	Year
Turing-NLG	17.2B	256 V100	—	Microsoft	2020
GPT-3	175B	—	—	OpenAI	2020
OPT-175B	175B	992 A100	2 months	Meta	2021
Gopher	280B	4096 TPU v3	1.3 months	Google	2021
MT-NLG	530B	4480 A100	3 months	Microsoft & NVIDIA	2022
PaLM	540B	6144 TPU v4	2 months	Google	2022
GPT-4	1.76T	—	4-7 months	OpenAI	2023

Larger training models

More GPUs involved

Longer training time

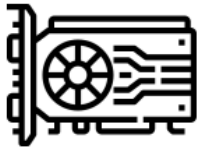
Failures are frequent

- Software failures

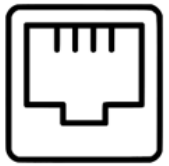
Library failures

Remote storage failures

- Hardware failures



GPU failures



Link failures

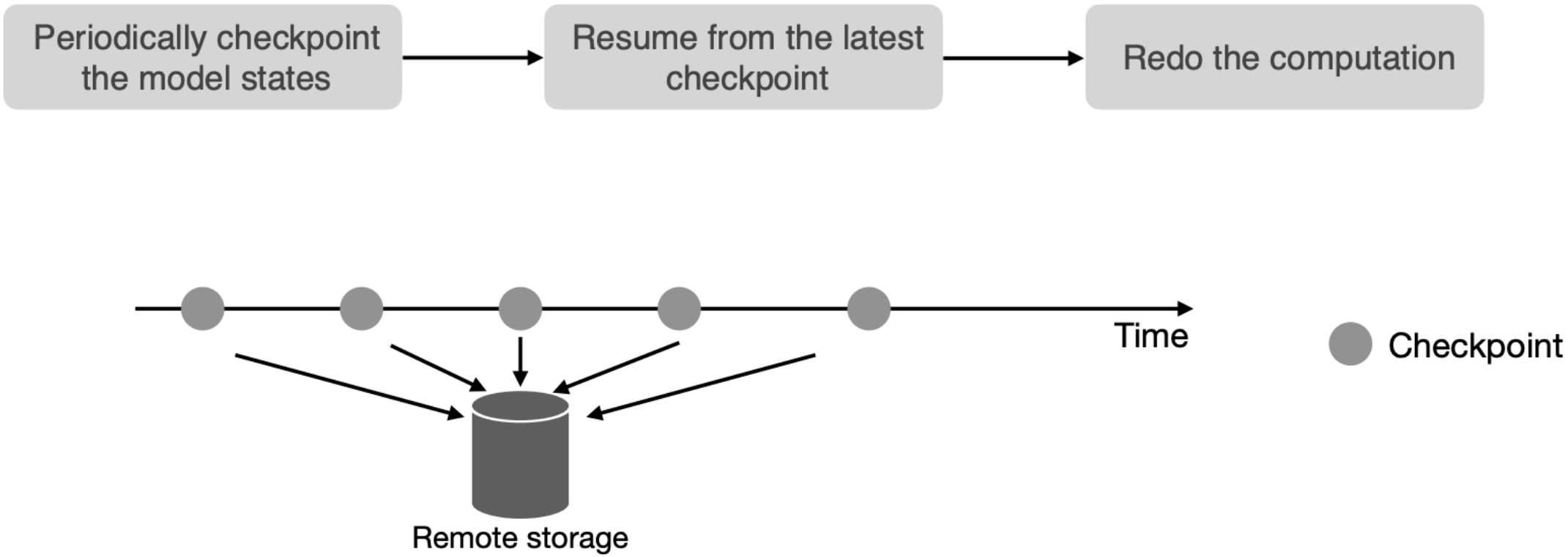
Switch failures

- OPT-175B: 100+ failures[1] in two months

[1] OPT paper

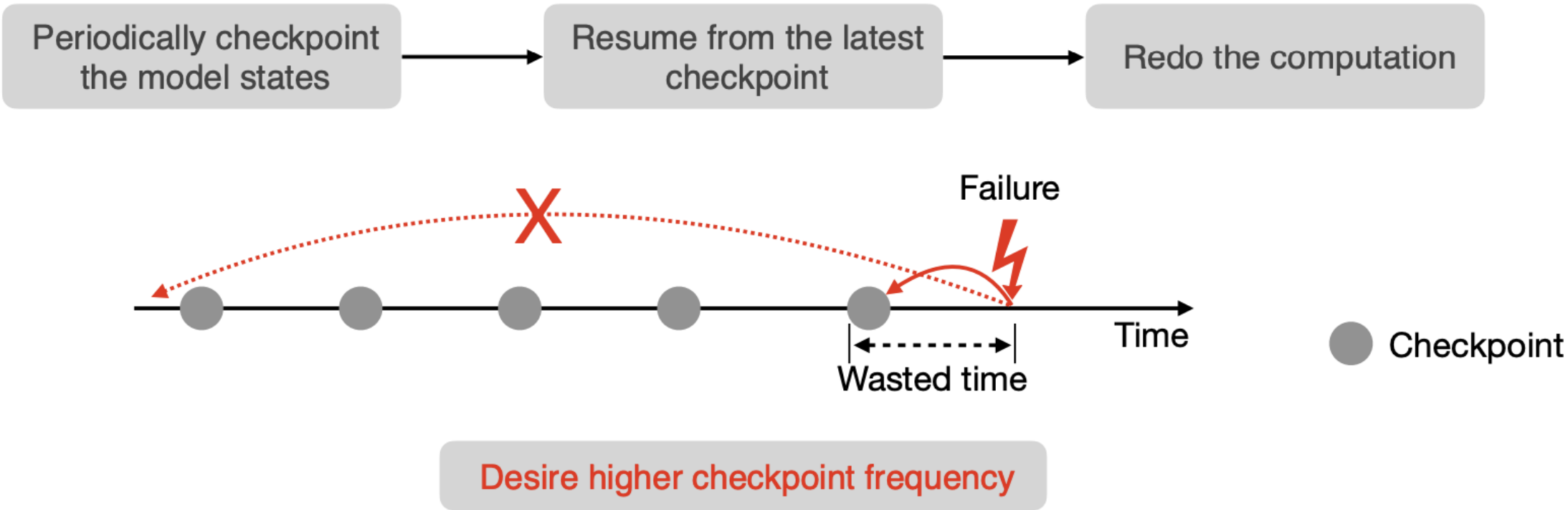
Checkpoint for failure recovery

- How checkpoint works?



Checkpoint for failure recovery

- How checkpoint works?



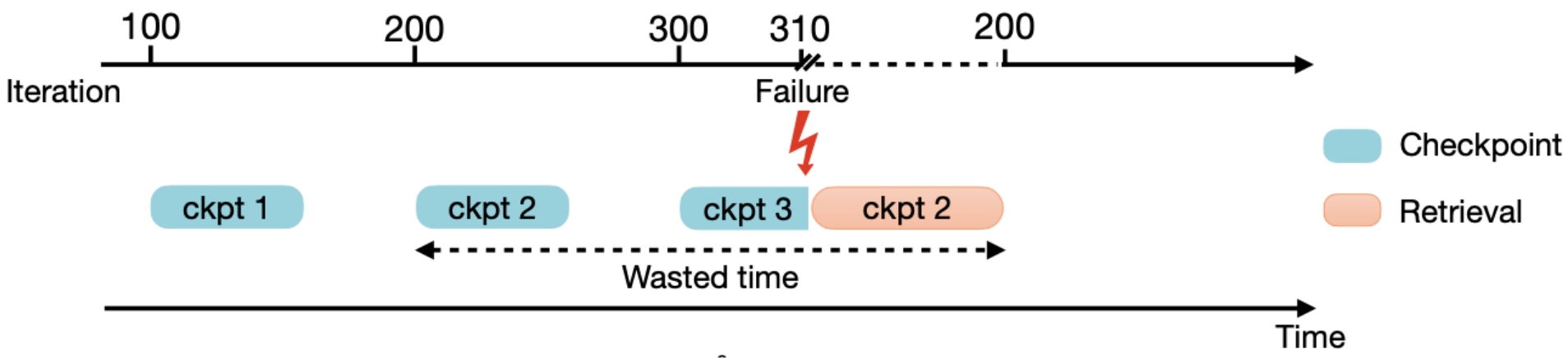
Checkpoint in LLM

Limited checkpoint frequency

- Checkpoint to remote storage takes a long time

Model	Parameters	Checkpoint size	Checkpoint time (20Gbps)
Gopher [56]	280B	3.4 TB	23 min
MT-NLG [62]	530B	6.4 TB	43 min
PaLM [23]	540B	6.5 TB	44 min

- Checkpoint frequency is limited by the checkpoint time



Checkpoint in LLM

Prohibitive failure recovery overhead

- Costly wasted time
  - Even with the highest checkpoint frequency

Model	Parameters	Checkpoint size	Checkpoint time (20Gbps)	Average wasted time
Gopher [56]	280B	3.4 TB	23 min	57 min
MT-NLG [62]	530B	6.4 TB	43 min	108 min
PaLM [23]	540B	6.5 TB	44 min	110 min

- Significant GPU resources are wasted due to failure recovery
  - Thousands of GPUs involved
  - Hundreds of failures during training

Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory

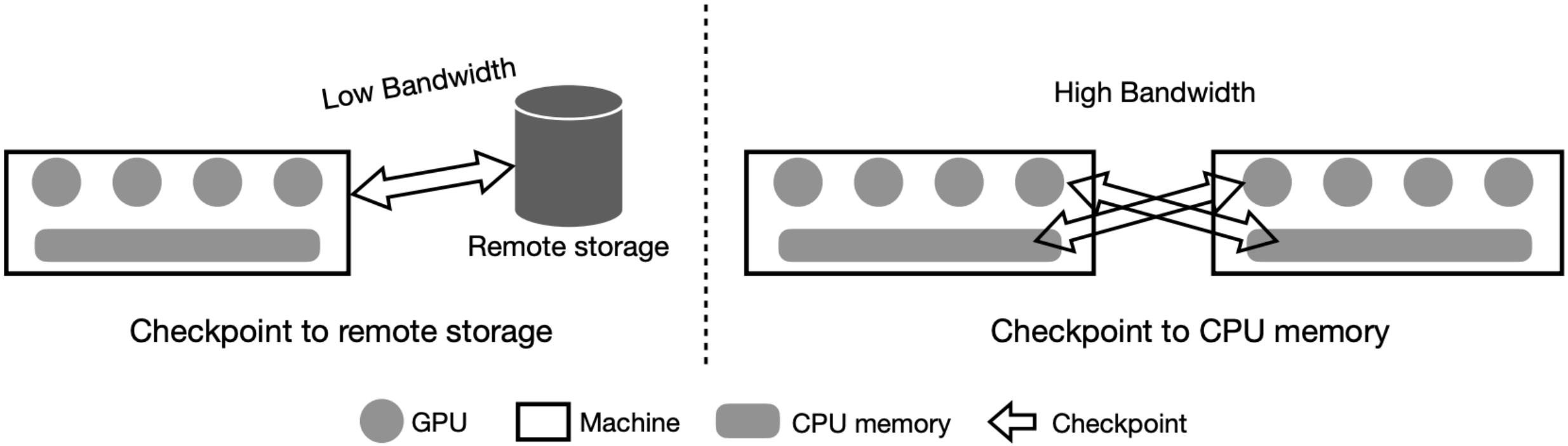
Instance type	Cloud	GPU	GPU memory	CPU memory
p3dn.24xlarge [14]	AWS	8 V100	8 × 32 GB	768 GB
p4d.24xlarge [15]	AWS	8 A100	8 × 40 GB	1152 GB
ND40rs_v2 [10]	Azure	8 V100	8 × 32 GB	672 GB
ND96asr_v4 [11]	Azure	8 A100	8 × 40 GB	900 GB
n1-8-v100 [9]	GCP	8 V100	8 × 32 GB	624 GB
a2-highgpu-8g [9]	GCP	8 A100	8 × 40 GB	640 GB
DGX A100 [12]	NVIDIA	8 A100	8 × 80 GB	2 TB

CPU memory size is sufficient to store checkpoints

Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory
- Checkpoint to CPU memory enables a much higher frequency

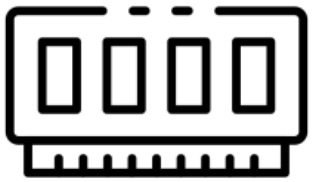


Gemini

Checkpoint to CPU memory

- CPU memory is much larger than GPU memory
- Checkpoint to CPU memory enables a much higher frequency
- CPU memory only stores checkpoints for failure recovery

Decouple the checkpoints for different purposes



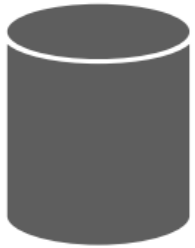
CPU memory

Failure recovery

- High-frequent checkpoints
- Only need the latest one

Debugging, accuracy evaluation

- Need checkpoint history
- Low-frequent checkpoints

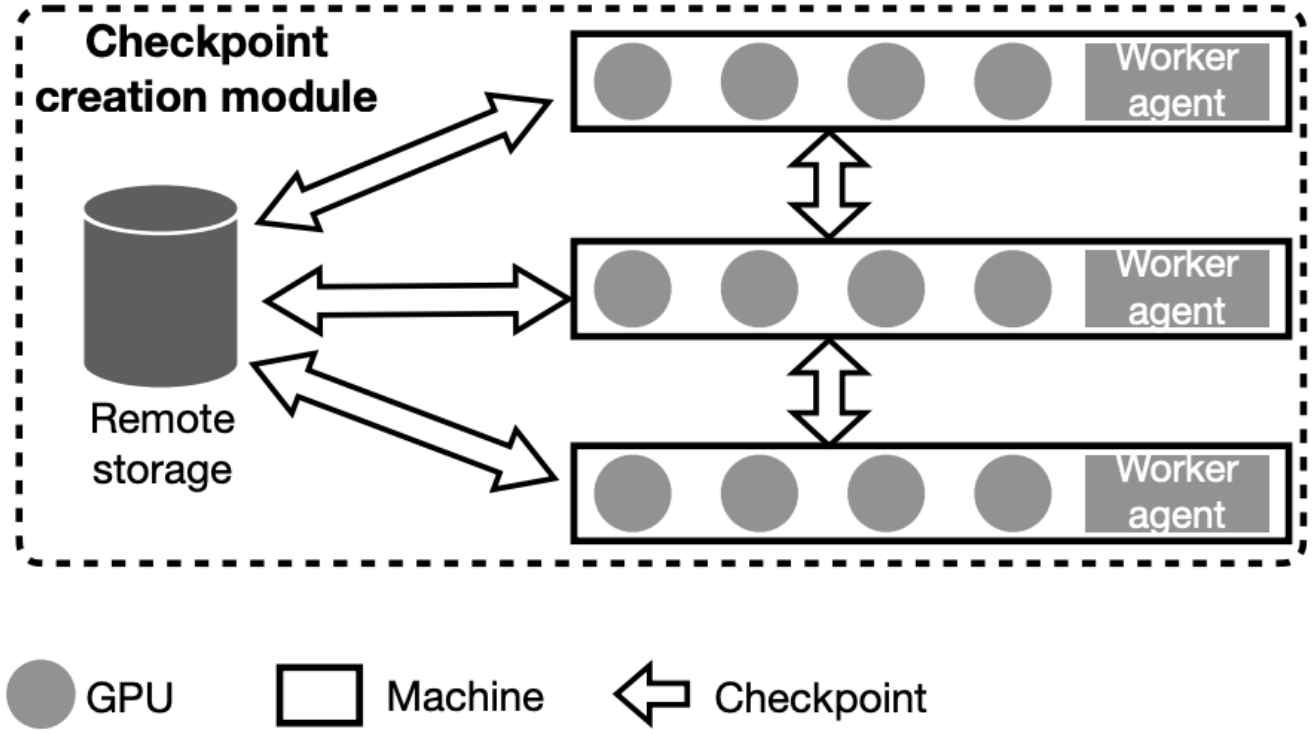


Remote storage

Gemini

Architecture

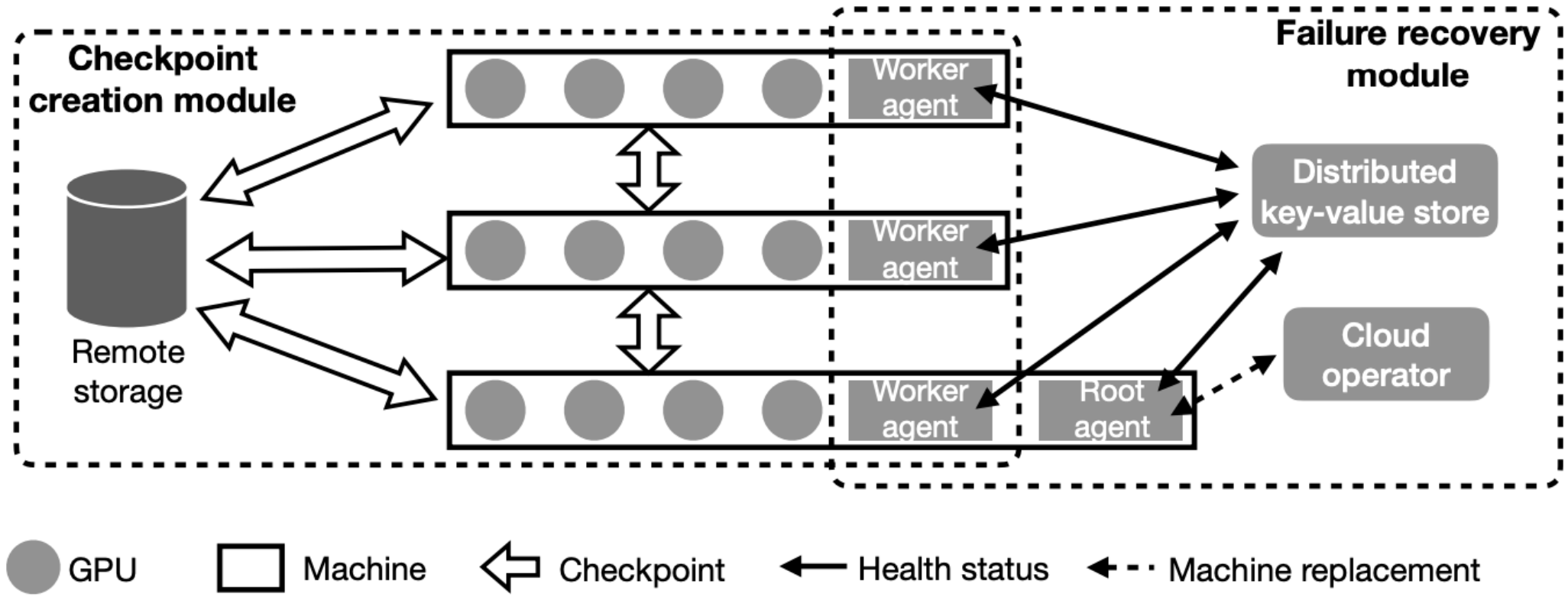
- Two modules



Gemini

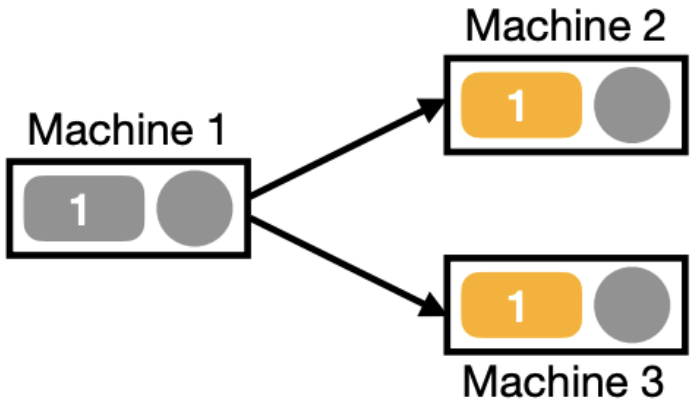
Architecture

- Two modules



Challenge #1

- Data stored in CPU memory can get lost
- Checkpoint redundancy
  - Design choice: checkpoint replicas



- Why not Erasure Coding?
  - Prohibitive computation cost
  - CPU memory is not a bottleneck

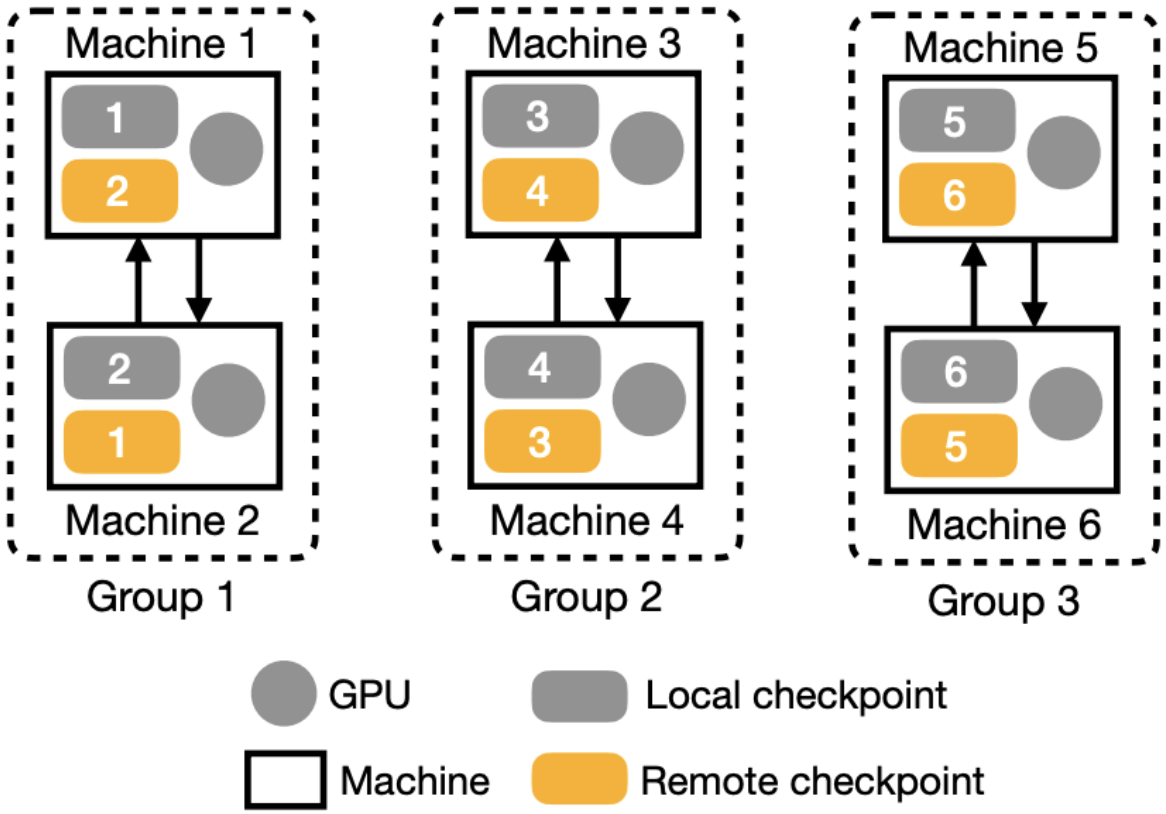
Solution

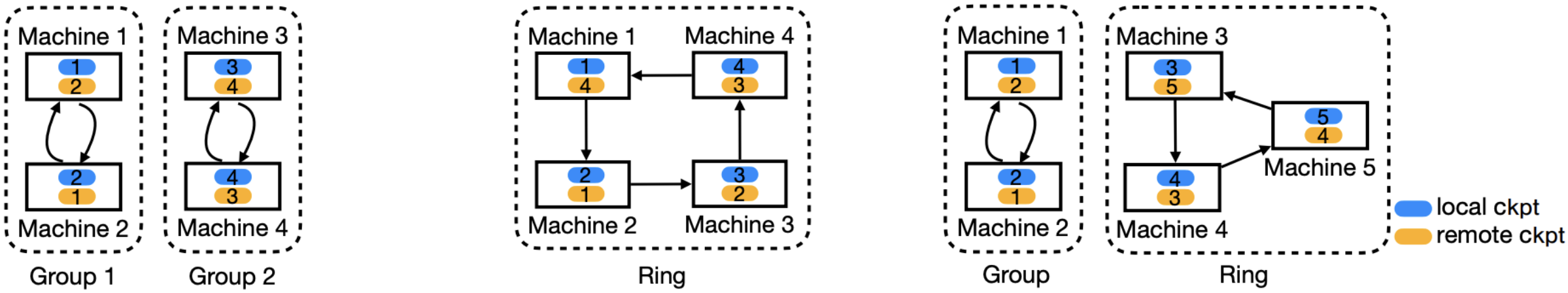
Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

- An example with two replicas





(a) Group placement strategy.

(b) Ring placement strategy.

(c) Mixed placement strategy.

Figure 3. Illustrations of the mixed checkpoint placement strategy.

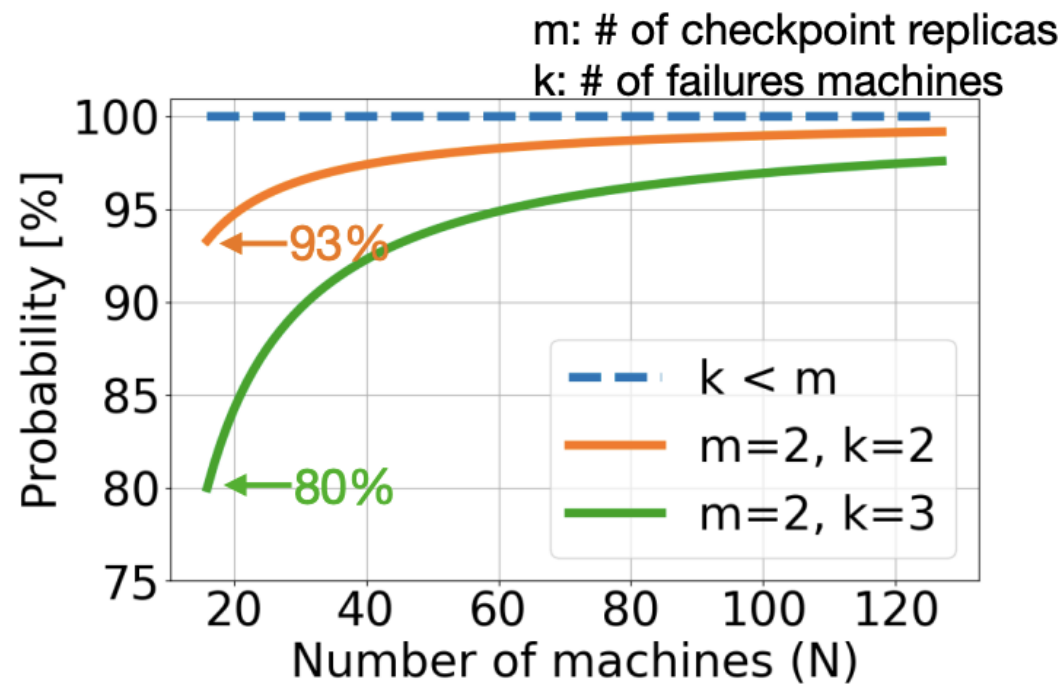
Solution

Group placement strategy

- Two steps

1. Given m replicas, all machines are divided into disjoint groups and each group has m machines
2. Each machine backups a checkpoint replica for all machines within the same group

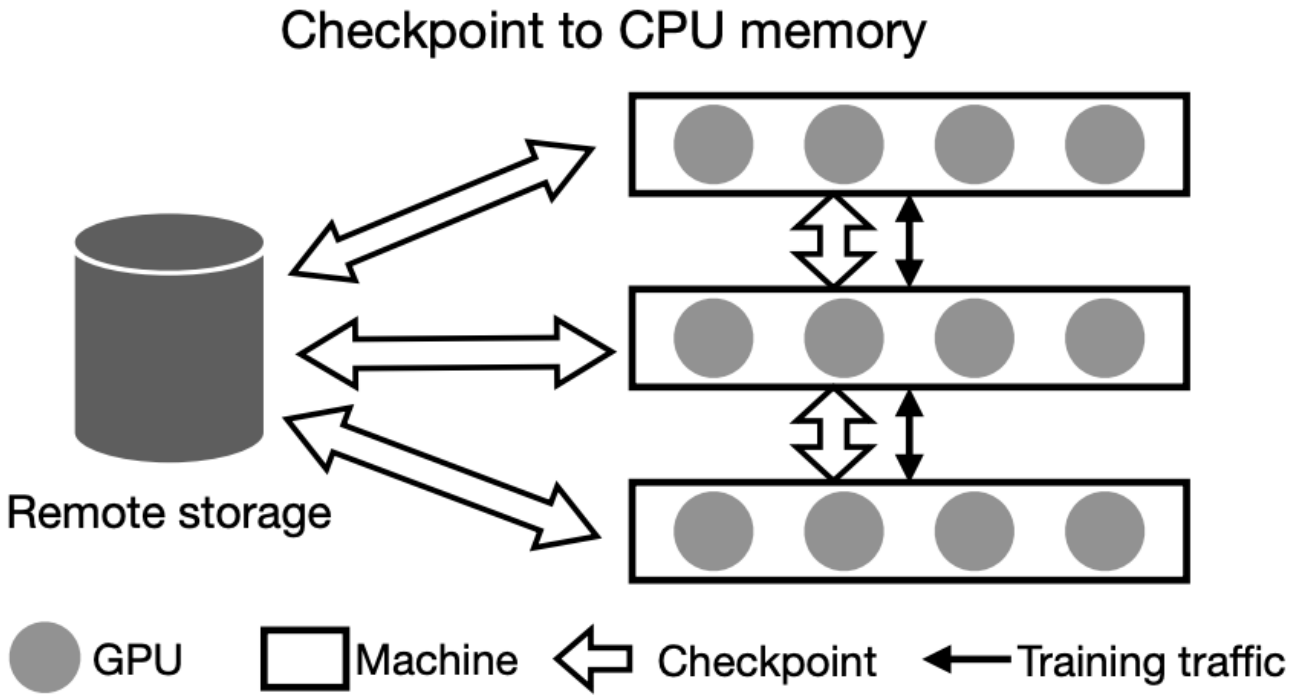
Group placement strategy is provably optimal



Two checkpoint replicas can already handle most failure cases!

Challenge #2

- Checkpoint traffic interferes with training traffic



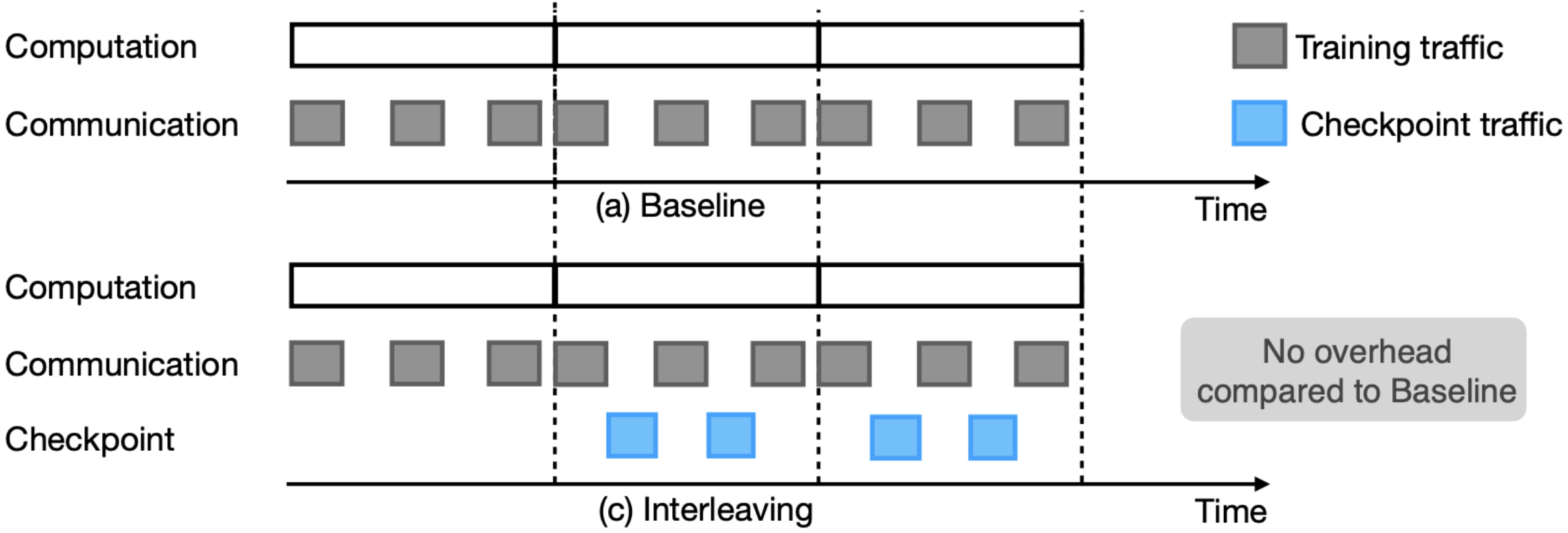
Checkpoint traffic and training traffic **shares the same network**

It can harm training throughput

Solution

Traffic interleaving

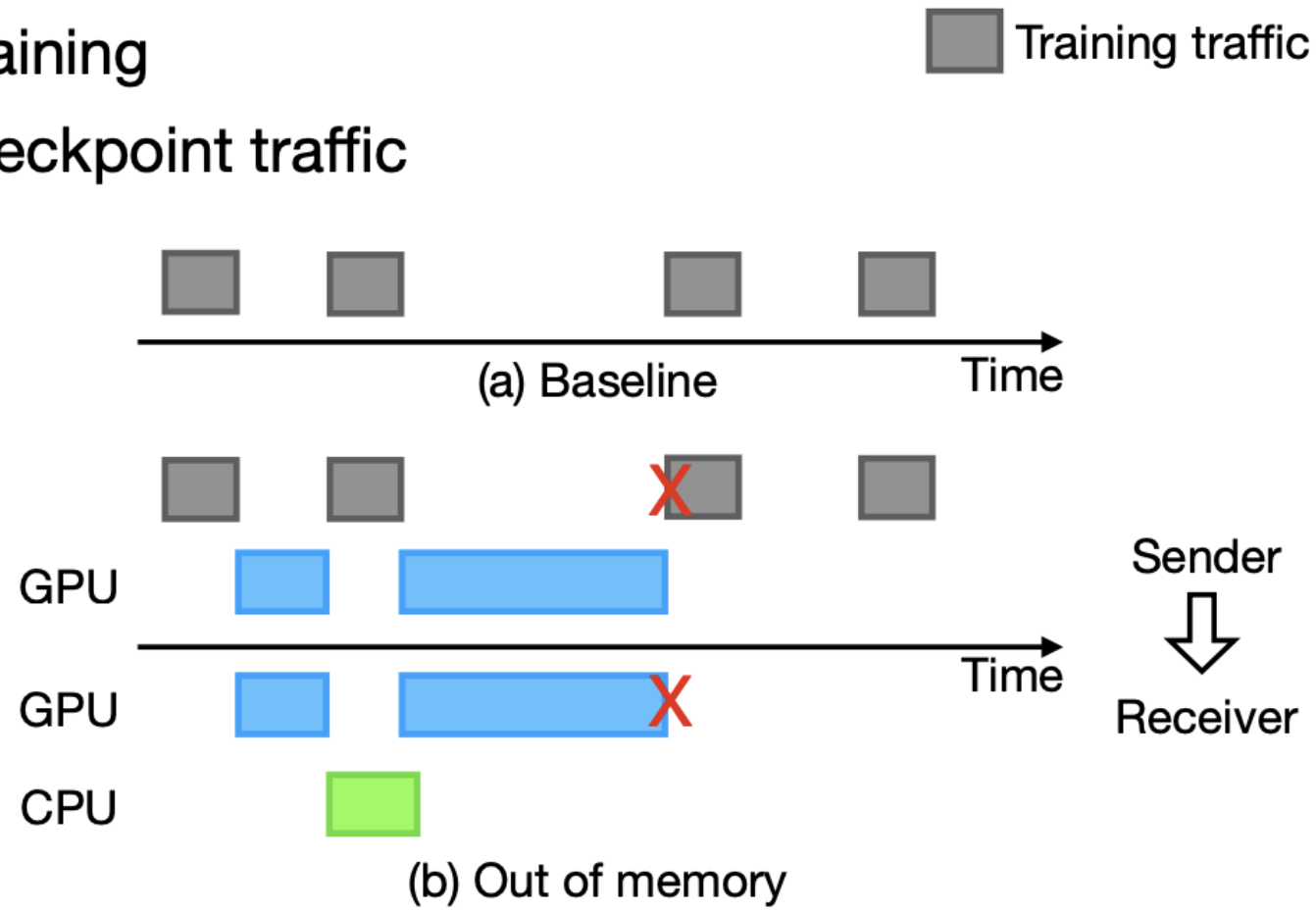
- Insert checkpoint traffic in idle timespans



Out-of-memory issue

- GPU memory is mainly used for training
- Limited spare GPU memory for checkpoint traffic

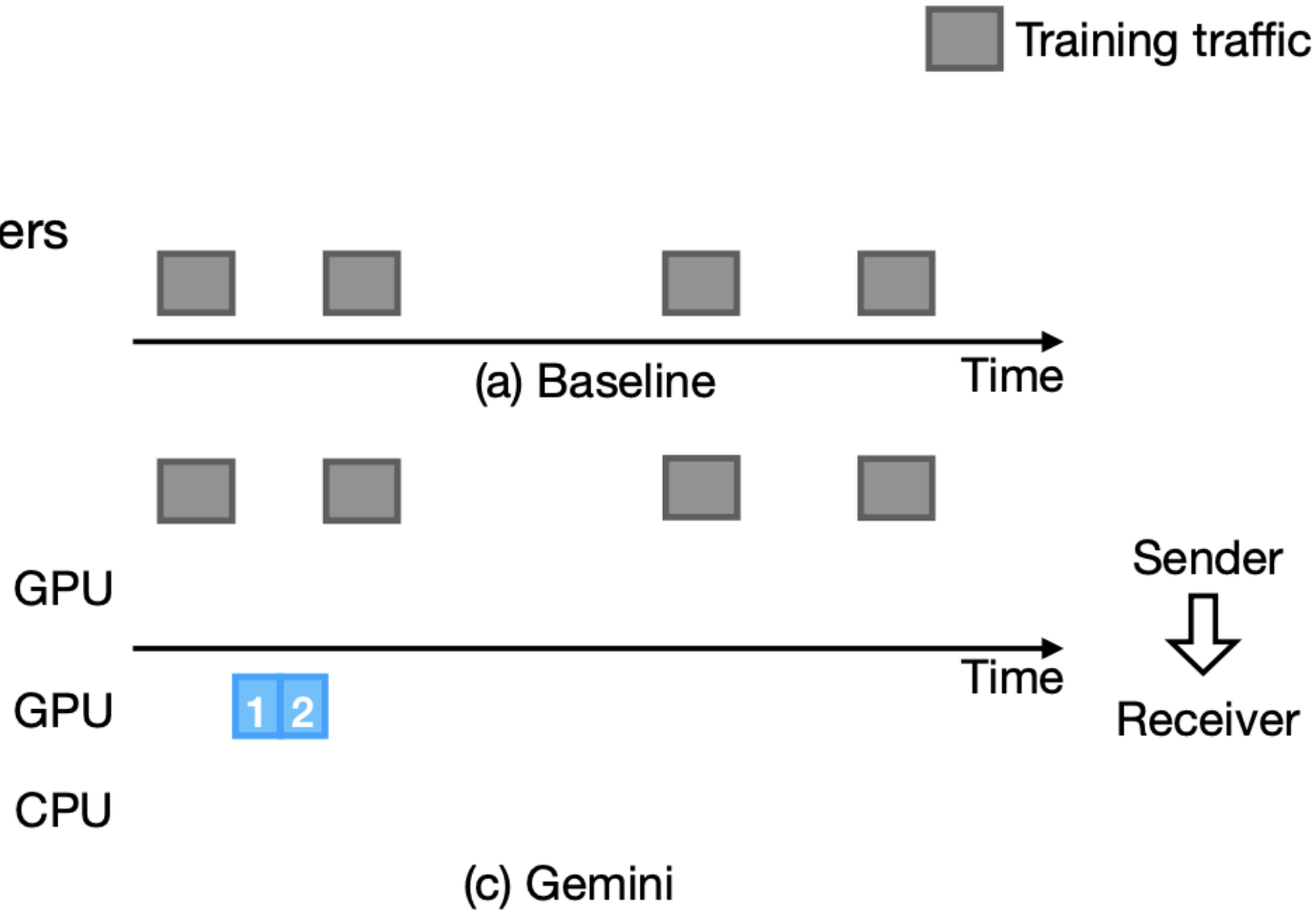
How to minimize the extra GPU memory consumption?



Our design

Checkpoint partition and pipelining

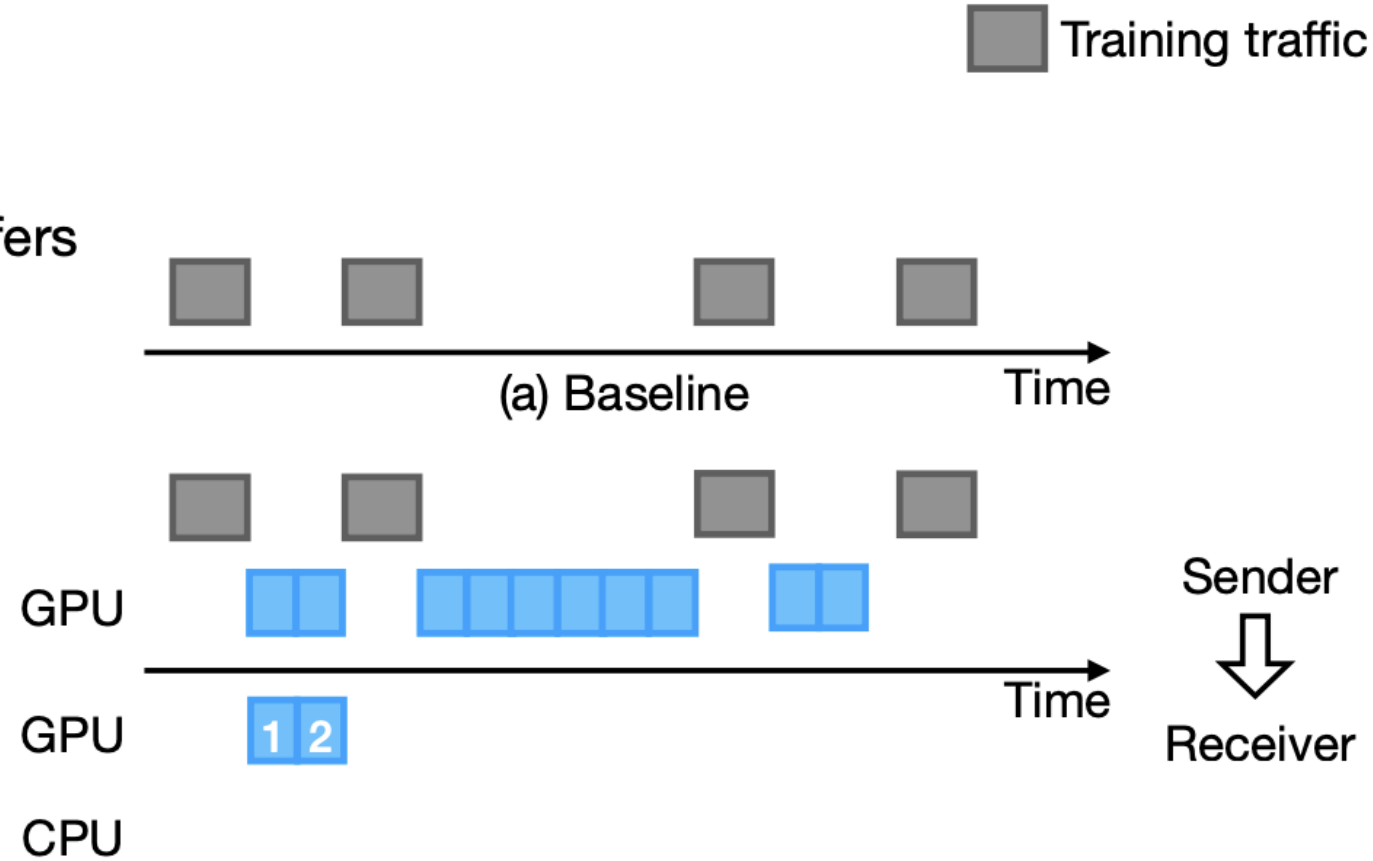
- Keys ideas
  - Reserve a GPU buffer at the receiver
  - Partition the buffer to multiple sub-buffers



Our design

Checkpoint partition and pipelining

- Keys ideas
  - Reserve a GPU buffer at the receiver
  - Partition the buffer to multiple sub-buffers
  - Pipeline checkpoint communications



(c) Gemini

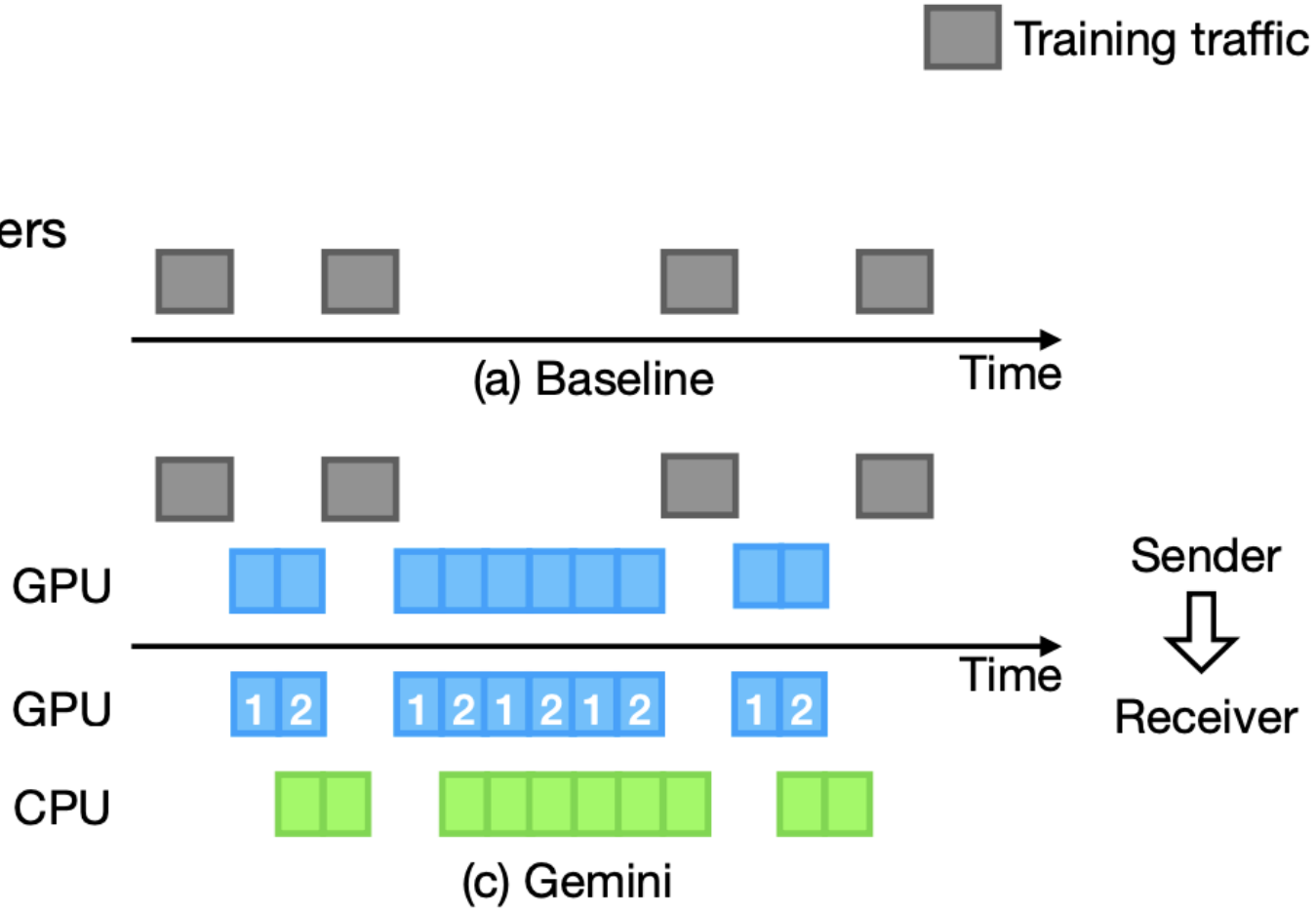
Our design

Checkpoint partition and pipelining

- Keys ideas
  - Reserve a GPU buffer at the receiver
  - Partition the buffer to multiple sub-buffers
  - Pipeline checkpoint communications

The GPU sub-buffers are reused

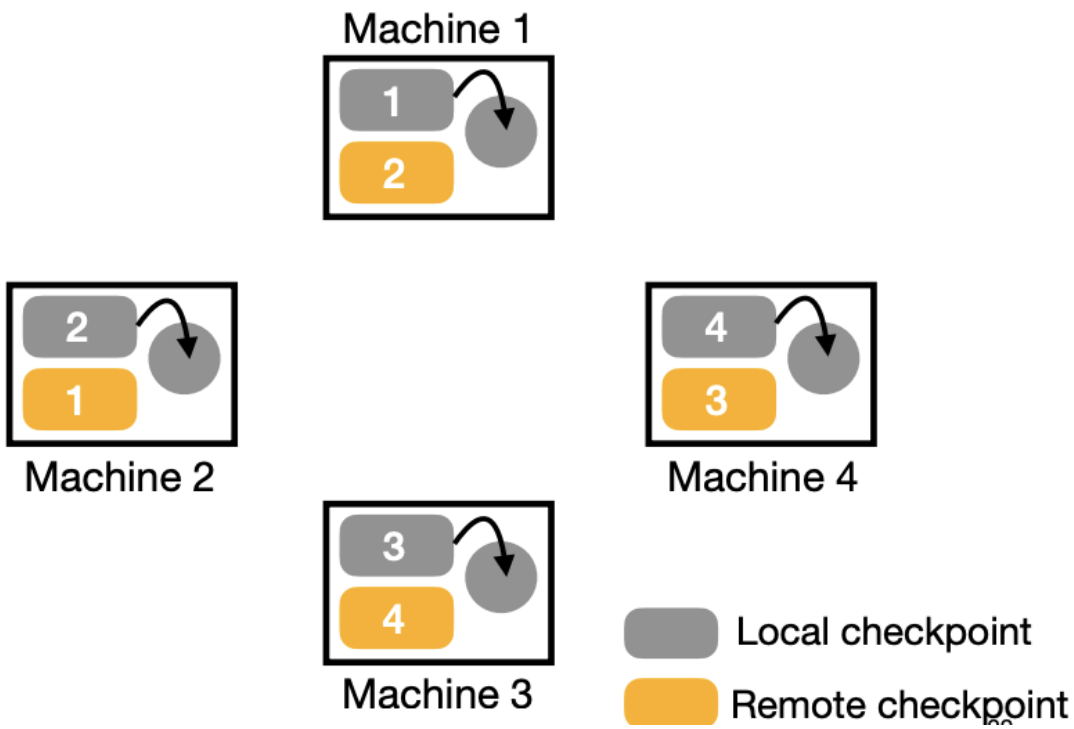
A small GPU buffer, e.g., 128MB, is sufficient



Resume training from failures

Software failures

- Checkpoints are available at local



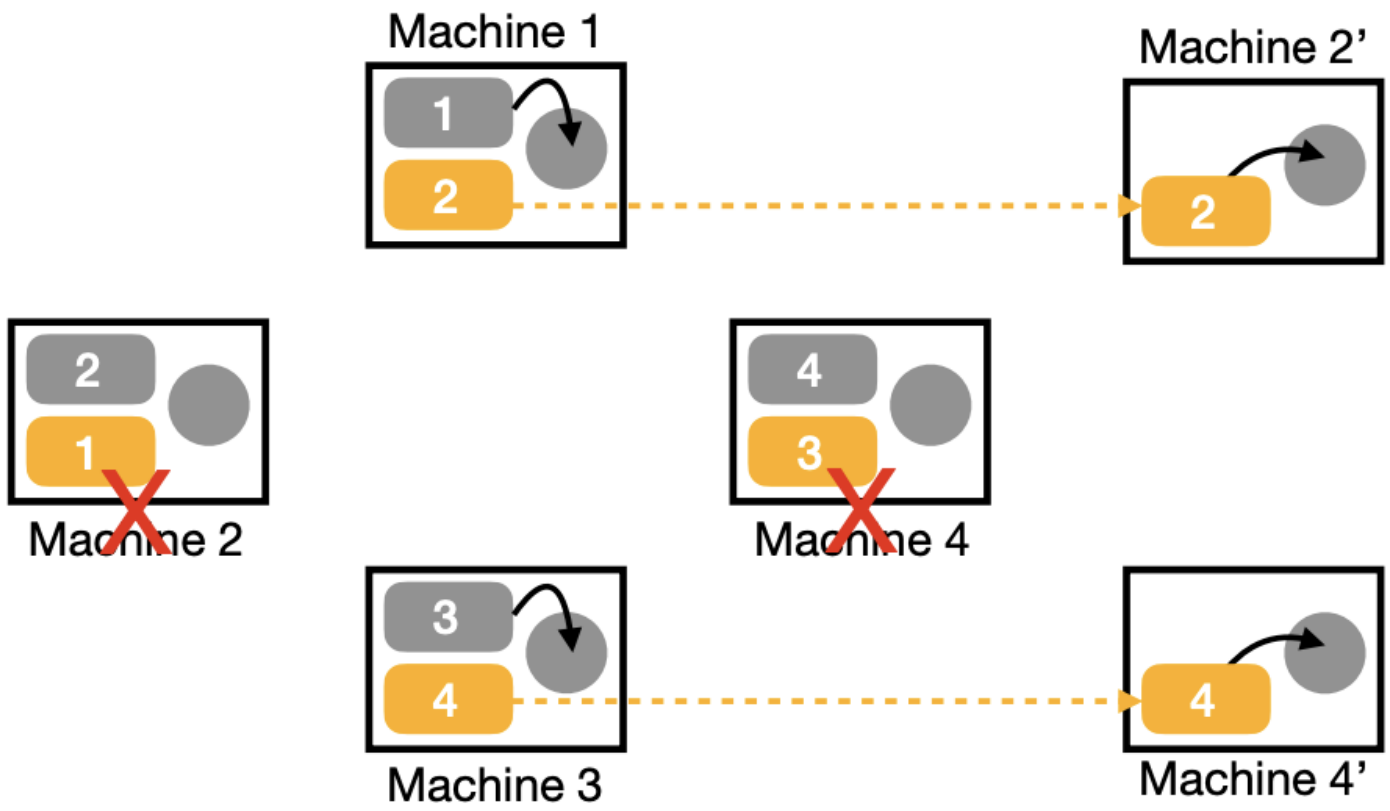
Negligible retrieval time

Just few iterations are lost

Resume training from failures

Hardware failures

- Checkpoints are still available at other machines

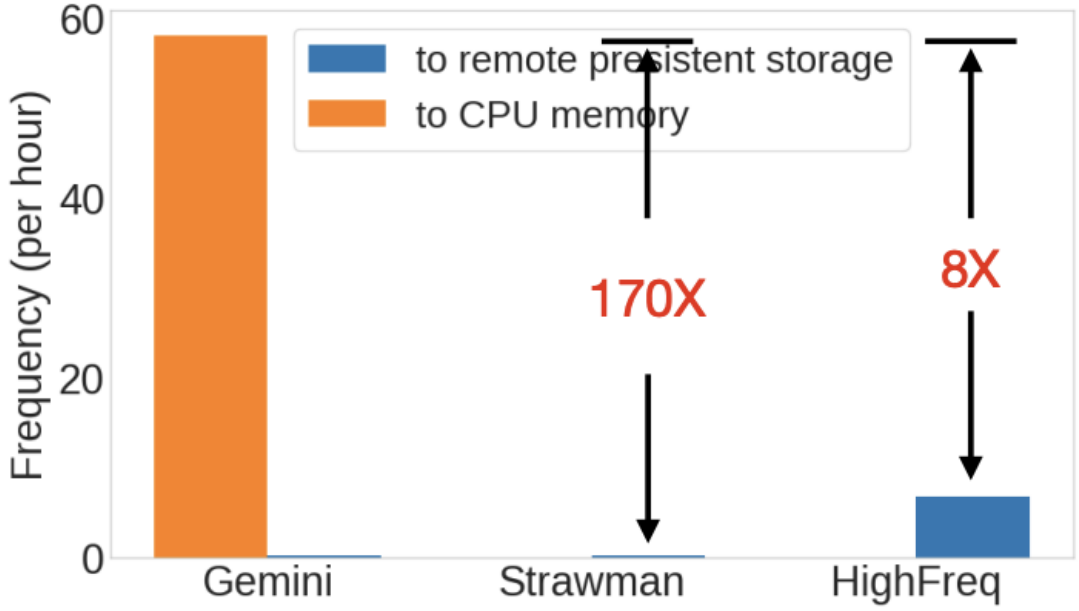


Evaluation

- Settings
  - Framework: DeepSpeed, ZeRO-3
  - 16 p4d instances (128 A100 GPUs), 400Gbps network bandwidth
  - The aggregated bandwidth of remote storage: 20Gbps
  - The size of LLM: 100 billion parameters
  - Reserved GPU buffer size: 128MB

Checkpoint frequency

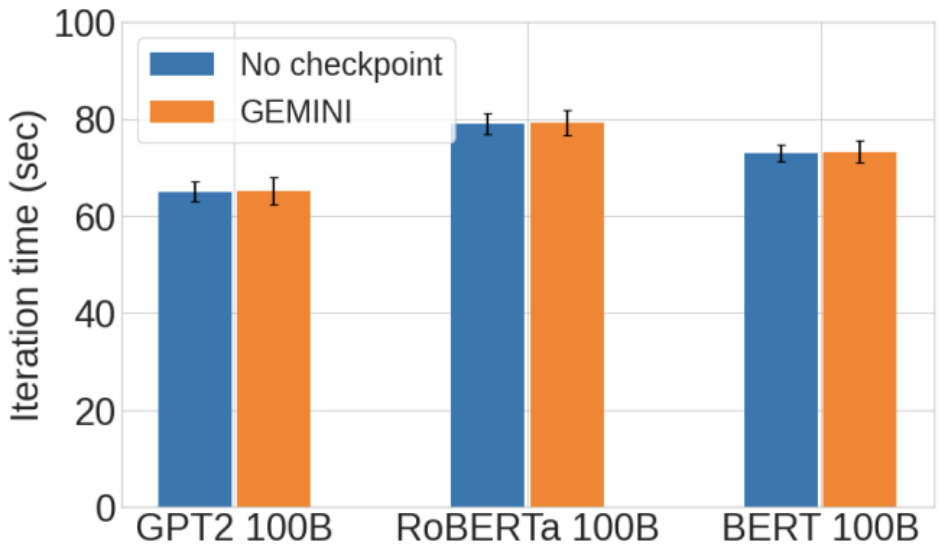
- Baselines
  - Strawman: every 3 hours (BLOOM's frequency [1])
  - HighFreq: saturate the remote storage bandwidth capacity



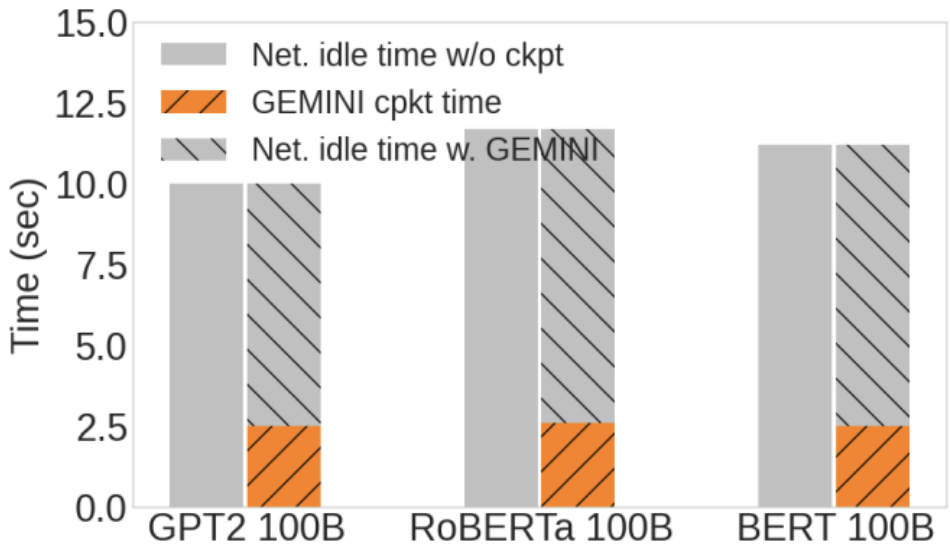
Gemini: Checkpoint model states every iteration

Training efficiency

- Training time
  - Gemini checkpoints model states to CPU memory every iteration



Negligible overhead on iteration time



Idle timespans can accommodate checkpoint traffic

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

DeepSeek-AI

research@deepseek.com

We introduce DeepSeek-V3.2, a model that harmonizes high computational efficiency with superior reasoning and agent performance. The key technical breakthroughs of DeepSeek-V3.2 are as follows: **(1) DeepSeek Sparse Attention (DSA)**: We introduce DSA, an efficient attention mechanism that substantially reduces computational complexity while preserving model performance in long-context scenarios. **(2) Scalable Reinforcement Learning Framework**: By implementing a robust reinforcement learning protocol and scaling post-training compute, DeepSeek-V3.2 performs comparably to GPT-5. Notably, our high-compute variant, DeepSeek-V3.2-Speciale, surpasses GPT-5 and exhibits reasoning proficiency on par with Gemini-3.0-Pro, achieving gold-medal performance in both the 2025 International Mathematical Olympiad (IMO) and the International Olympiad in Informatics (IOI). **(3) Large-Scale Agentic Task Synthesis Pipeline**: To integrate reasoning into tool-use scenarios, we developed a novel synthesis pipeline that systematically generates training data at scale. This methodology facilitates scalable agentic post-training, yielding substantial improvements in generalization and instruction-following robustness within complex, interactive environments.

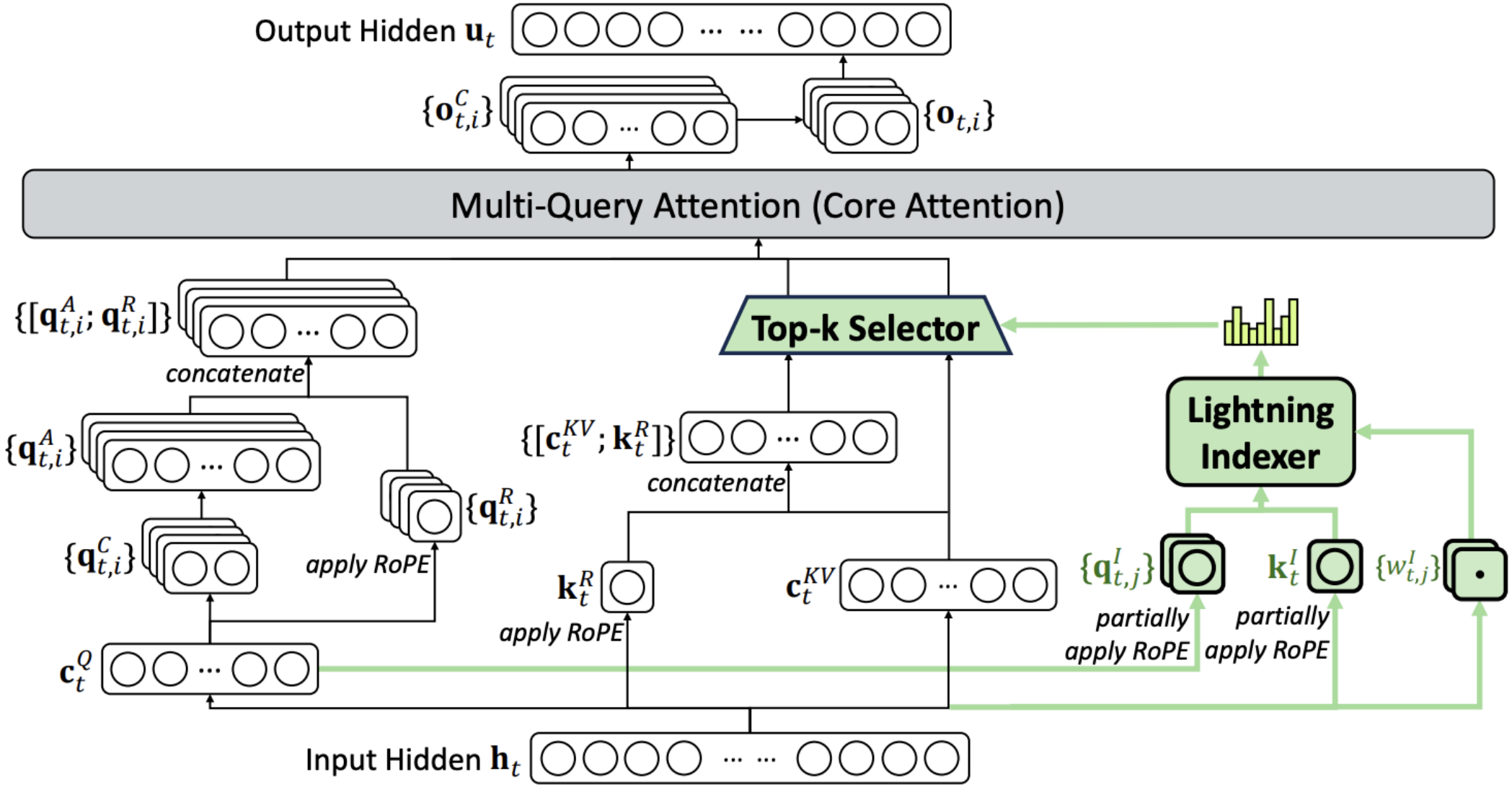


Figure 2 | Attention architecture of DeepSeek-V3.2, where DSA is instantiated under MLA. The green part illustrates how DSA selects the top-k key-value entries according to the indexer.

Prototype of DSA. The prototype of DSA primarily consists of two components: a lightning indexer and a fine-grained token selection mechanism.

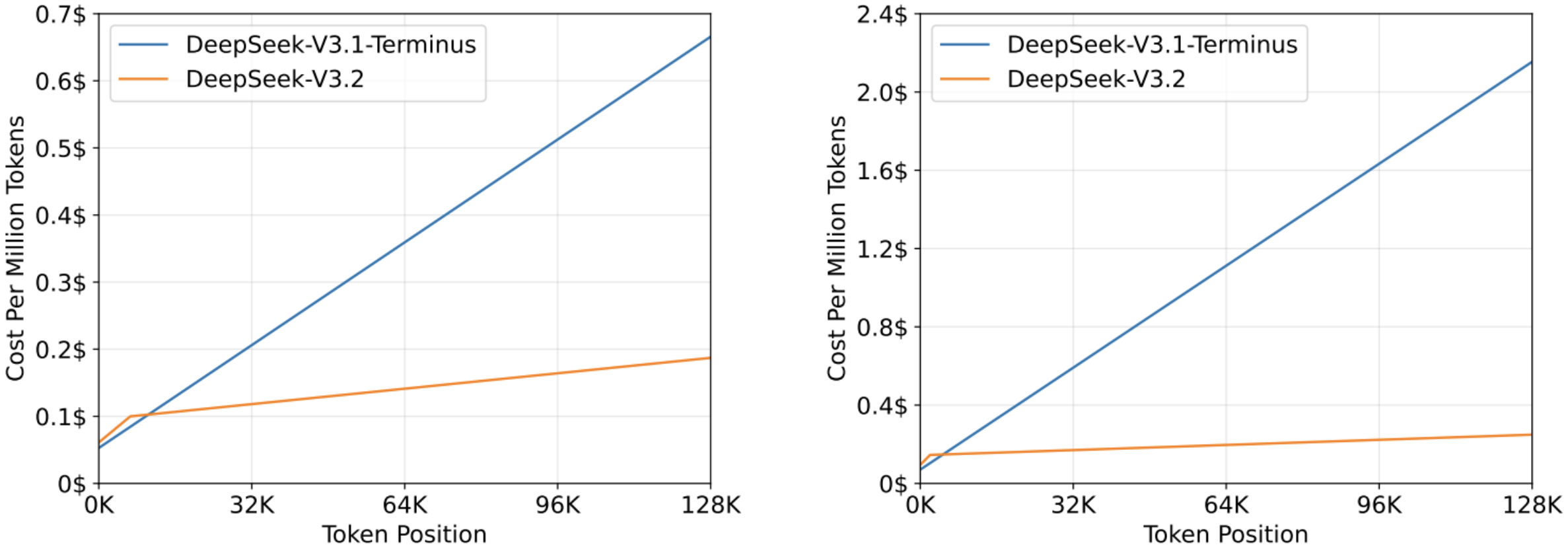
The **lightning indexer** computes the index score $I_{t,s}$ between the query token $\mathbf{h}_t \in \mathbb{R}^d$ and a preceding token $\mathbf{h}_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \mathrm{ReLU}\left(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I\right), \tag{1}$$

where H^I denotes the number of indexer heads; $\mathbf{q}_{t,j}^I \in \mathbb{R}^{d^I}$ and $w_{t,j}^I \in \mathbb{R}$ are derived from the query token $\mathbf{h}_t$; and $\mathbf{k}_s^I \in \mathbb{R}^{d^I}$ is derived from the preceding token $\mathbf{h}_s$. We choose ReLU as the activation function for throughput consideration. Given that the lightning indexer has a small number of heads and can be implemented in FP8, its computational efficiency is remarkable.

Given the index scores $\{I_{t,s}\}$ for each query token $\mathbf{h}_t$, our **fine-grained token selection mechanism** retrieves only the key-value entries $\{\mathbf{c}_s\}$ corresponding to the top-k index scores. Then, the attention output $\mathbf{u}_t$ is computed by applying the attention mechanism between the query token $\mathbf{h}_t$ and the sparsely selected key-value entries $\{\mathbf{c}_s\}$:

$$\mathbf{u}_t = \mathrm{Attn}\left(\mathbf{h}_t, \left\{\mathbf{c}_s \,\middle|\, I_{t,s} \in \text{Top-k}(I_{t,:})\right\}\right). \tag{2}$$



(a) Prefilling

(b) Decoding

Figure 3 | Inference costs of DeepSeek-V3.1-Terminus and DeepSeek-V3.2 on H800 clusters.